

Lecture 16

Quantization for AI/NPU Compiler

INT8/INT4 requantization, accumulator width, and MLIR quant lowering

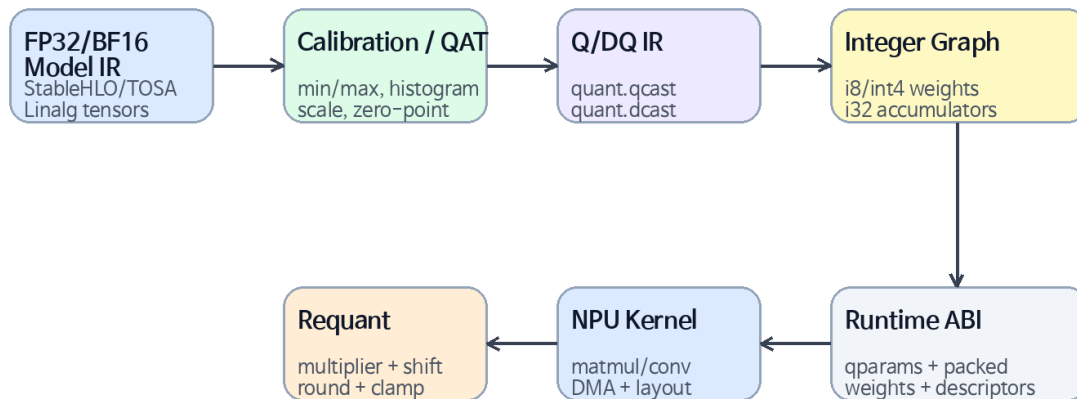
강의 위치	핵심 개념	실습 산출물
16/20 – Quantization	scale, zero-point, qcast/dcast, rescale, INT8/INT4	NPU quantization policy + verifier/test

Learning Outcomes

- Uniform quantization 수식을 MLIR type/operation contract 로 해석한다.
- !quant.uniform, quant.qcast, quant.dcast, quant.scast 를 읽고 NPU lowering boundary 를 설계한다.
- INT8 MatMul accumulator, bias scale, requant multiplier/shift 를 계산한다.
- INT4 weight packing, group scale, memory alignment policy 를 정의한다.
- overflow, rounding, saturation, qparams mismatch 를 잡는 verifier 와 regression test 를 설계한다.

전체 흐름

Quantization Compiler Flow for NPU



Key compiler contract: keep floating semantics long enough to choose qparams, then lower to legal integer kernels with explicit rounding and saturation.

1. 왜 Quantization OI NPU Compiler 의 핵심인가?

Physical AI 용 edge/embedded NPU 는 TOPS/W, SRAM footprint, DRAM bandwidth, thermal envelope 가 모두 제한됩니다. Quantization 은 FP32/BF16 연산을 INT8/INT4 같은 낮은 bit-width 정수 연산으로 바꿔 MAC density 와 memory bandwidth 효율을 높이는 대표적인 방법입니다. 그러나 compiler 가 scale/zero-point, accumulator width, rounding, saturation, bias scale, per-channel metadata 를 정확히 관리하지 못하면 정확도 손실이나 overflow 가 발생됩니다.

NPU compiler 에서 quantization 은 단순 dtype 변환이 아닙니다. 모델 import 단계의 floating semantics, calibration/QAT 결과, tensor layout, weight packing, DMA format, kernel ABI, runtime descriptor 까지 연결되는 cross-layer contract 입니다.

2. Uniform Quantization 수식

MLIR Quantization 문서는 real value, fixed-point/scaled value, affine value 의 관계를 다음처럼 설명합니다.

$$\text{real_value} = \text{scaled_value} * \text{scale} = (\text{affine_value} - \text{zero_point}) * \text{scale}$$

일반적인 affine quantization 에서는 floating value x 를 정수 q 로 변환할 때 다음 형태를 사용합니다.

$$q = \text{clamp}(\text{round}(x / \text{scale}) + \text{zero_point})$$

$$x_{\text{hat}} = (q - \text{zero_point}) * \text{scale}$$

여기서 scale 은 인접 quantized code 간 real-space 간격이고, zero_point 는 real zero 를 정확히 표현하기 위한 integer code 입니다. Convolution/MatMul 에서 zero padding 이나 bias-free zero 가 중요하기 때문에 zero 가 정확히 표현되지 않으면 output bias 가 생길 수 있습니다.

3. MLIR Quant Dialect 읽기

MLIR 의 quant dialect 는 expressed value 와 stored value 사이의 mapping 을 type 으로 encode 합니다. 예를 들어 `!quant.uniform<i8:f32, 0.03125:-3>` 는 f32 expressed value 를 i8 stored value 로 저장하며 scale=0.03125, zero_point=-3 을 사용한다는 의미입니다.

중요 operation 은 세 가지입니다.

quant.qcast : expressed floating value → quantized type
 quant.dcast : quantized type → expressed floating value
 quant.scast : quantized type <-> integer storage type reinterpretation

NPU compiler 에서는 qcast/dcast pair 를 Q/DQ boundary 로 보고, legalizable subgraph 를 찾아 integer kernel 로 치환합니다. qcast/dcast 를 너무 일찍 모두 integer arithmetic 으로 낮추면 fusion, layout propagation, fallback boundary 추적이 어려워질 수 있습니다.

4. Per-tensor, Per-channel, Blockwise Quantization

Per-tensor quantization 은 전체 tensor 에 하나의 scale/zero-point 를 적용합니다. Activation 에는 보통 per-tensor 가 간단하고 runtime overhead 가 낮습니다.

Per-channel quantization 은 channel axis 별로 scale/zero-point 를 따로 둡니다. Weight quantization 에서는 output channel 마다 dynamic range 가 다르기 때문에 per-output-channel scale 이 정확도 측면에서 유리합니다. Compiler 는 channel axis 가 layout transform 이후에도 일관되게 유지되는지 검증해야 합니다.

Blockwise 또는 sub-channel quantization 은 tensor block/group 단위로 scale 을 둡니다. INT4 weight-only quantization 에서 group_size=32/64/128 같은 정책과 자주 연결됩니다. 이 경우 runtime descriptor 와 packed

weight layout 이 qparams layout 과 정확히 맞아야 합니다.

5. INT8 MatMul Quantization 수식

INT8 MatMul 에서 activation qa, weight qw, output qy 를 가정하면 accumulator 는 보통 i32 입니다.

$$\text{acc_i32}[m,n] = \sum_k (qa[m,k] - za) * (qw[k,n] - zw[n])$$

real output 은 다음과 같습니다.

$$y_{\text{real}} = \text{scale_a} * \text{scale_w}[n] * \text{acc_i32} + \text{bias_real}[n]$$

output quantized value 는 다음과 같이 requantize 합니다.

$$qy = \text{clamp_i8}(\text{round}((\text{scale_a} * \text{scale_w}[n] / \text{scale_y}) * \text{acc_i32} + \text{bias_scaled}[n]) + zy)$$

여기서 per-channel weight scale 을 쓰면 n 또는 output-channel 별로 multiplier/shift 가 달라집니다. Compiler 는 bias scale 이 $\text{scale_a} * \text{scale_w}[n]$ 와 일치하는지, bias dtype 이 i32 인지, output requant scale 이 target kernel 에서 지원되는지 검증해야 합니다.

6. Requantization: multiplier + shift

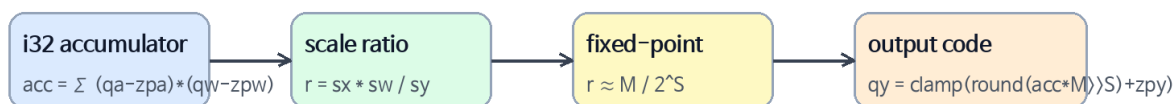
NPU 하드웨어는 floating multiply 를 매 output element 마다 실행하지 않습니다. scale ratio $r = \text{scale_a} * \text{scale_w} / \text{scale_y}$ 를 정수 multiplier M 과 shift S 로 근사합니다.

$$r \approx M / 2^S$$

$$qy = \text{saturate}(\text{rounding_shift}(\text{acc_i32} * M, S) + zy)$$

TOSA 의 tosa.rescale 은 integer multiply, add, shift 기반 rescale 을 표현하며 multiplier/shift, input/output zero point, per_channel 여부, rounding mode 같은 속성을 명시합니다. NPU dialect 로 낮출 때도 rounding mode 와 saturation range 가 target ABI 에 반드시 남아야 합니다.

Requantization: i32 accumulator → i8 output



```

real_y = sy * (qy - zpy)
real_acc = sx * sw * acc
qy = saturate_i8( round(acc * sx * sw / sy) + zpy )
fixed-point: qy = saturate_i8( rounding_shift(acc * M, S) + zpy )
  
```

NPU HW contract: rounding mode, saturation range, per-channel multiplier source, and accumulator bit-width must be explicit in IR/AB

7. Accumulator Width 분석

정수 MAC 의 output 정확도와 안전성은 accumulator width 가 좌우합니다. worst-case bound 는 다음처럼 계산할 수 있습니다.

$$\begin{aligned} \text{max_abs_acc} &\approx K * \text{max_abs_a} * \text{max_abs_w} + \text{bias_margin} \\ \text{required_bits} &\approx 1 + \text{ceil}(\log_2(\text{max_abs_acc} + 1)) \end{aligned}$$

대칭 INT8 에서는 $\text{max_abs_a}=\text{max_abs_w}=127$ 이므로 product bound 는 16129 입니다. i32 signed accumulator 의 양수 최대값 2,147,483,647 을 기준으로 K 는 약 133k 까지 worst-case 안전합니다. affine uint8 계열에서 zero-point 보정 범위를 255 로 잡으면 product bound 65025 가 되어 K safe bound 는 약 33k 입니다. 실제 network 는 더 작을 수 있지만 verifier 와 hardware contract 는 worst-case 또는 profiled guarantee 중 무엇을 쓰는지 명확히 해야 합니다.

Accumulator Width Analysis

case	operand range	max product	K safe in i32
int8 sym	$[-127, 127] \times [-127, 127]$	16,129	~133k
uint8 affine	$[-255, 255] \times [-255, 255]$	65,025	~33k
int4 sym	$[-7, 7] \times [-7, 7]$	49	~43M
int4 unsigned	$[0, 15] \times [0, 15]$	225	~9.5M

Required bits $\approx 1 + \text{ceil}(\log_2(K * \text{max_abs_a} * \text{max_abs_w} + \text{bias_margin} + 1))$

NPU compiler must reject/retile/fallback when K, zero-point correction, or bias add can overflow target accumulator.

주의: 표는 worst-case bound 입니다. 실제 모델은 calibration/statistics로 더 작을 수 있지만 verifier는 worst-case 또는 profiled contract를 명확히 해야 합니다.

8. INT4 Weight Quantization

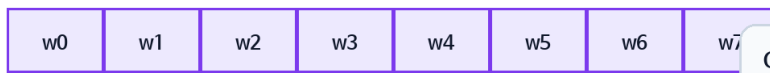
INT4 는 activation 까지 INT4 로 내리는 full INT4 와 weight 만 INT4 로 저장하고 activation 은 INT8/BF16 으로 유지하는 weight-only 방식으로 나눌 수 있습니다. Edge NPU 에서는 weight bandwidth 를 줄이기 위해 weight-only INT4 가 특히 중요합니다.

Compiler 관점의 핵심은 packing 입니다. 두 개의 4-bit weight 를 한 byte 에 넣을 때 low/high nibble 순서, signed s4 vs unsigned u4 해석, group_size, scale layout, K alignment, DMA burst alignment 가 모두 target contract 가 됩니다.

예: `rhs_packing="two_per_byte_low_first", rhs_type="s4", group_size=64, scale_layout="per_out_channel_per_group".`

INT4 Weight Quantization and Packing

Logical int4 weights



Packed bytes in DRAM/SRAM



Compiler checks:

- signedness: s4 vs u4
- nibble order: low/high
- group size: e.g. 32/64/128
- scale granularity: per-channel or per-group
- K alignment for tensor core
- unpack path vs native int4 MAC

Pseudo IR:

```
npu_kernel.qmatmul {lhs_type="i8", rhs_type="s4",
  rhs_packing="two_per_byte_low_first", group_size=64,
  scale_layout="per_out_channel_per_group"}
```

9. NPU Quantization Pipeline

권장 lowering 흐름은 다음과 같습니다.

FP32/BF16 input IR

- > calibration/QAT statistics import
- > qparams assignment
- > qcast/dcast insertion or recognition
- > QDQ propagation and cleanup
- > integerizable subgraph detection
- > INT8/INT4 kernel legalization
- > requant multiplier/shift materialization
- > weight packing and qparams layout
- > npu_kernel.qmatmul/qconv lowering
- > runtime descriptor emission

Pipeline 에서 중요한 원칙은 QDQ boundary 를 유지해 fallback 을 추적하고, integer kernel 로 바꾸기 전에는 numeric contract 를 명확히 검증하는 것입니다.

10. Verification Checklist

NPU compiler 의 quantization verifier 는 다음 항목을 확인해야 합니다.

- scale > 0, zero_point 가 storage range 안에 있음
- per-channel scale 수와 channel dimension 이 일치함
- layout transform 이후 channel axis 가 정확히 remapped 됨
- accumulator width 가 K와 operand range 에 대해 안전함
- bias dtype 과 bias scale 이 input*weight scale 과 일치함
- requant multiplier/shift 가 target 범위 안에 있음
- rounding mode 와 saturation semantics 가 target kernel 과 일치함
- INT4 packing order 와 group scale layout 이 runtime descriptor 와 일치함
- negative tests 로 unsupported qparams, overflow, misaligned group size 를 잡음

11. 실습 산출물

이번 강의의 산출물은 다음과 같습니다.

1. INT8 MatMul quantization 수식 annotation
2. !quant.uniform type, qcast/dcast/scast 예제 작성
3. tosa.rescale 기반 requant pseudo IR 작성
4. accumulator width 와 K bound 계산
5. INT4 weight packing contract 정의
6. NPU quantization policy YAML 작성
7. FileCheck positive/negative test skeleton 작성

NPU Quantization Policy Template

```
target: edge_npu_v1
activation:
  type: i8
  granularity: per_tensor
  symmetric: false
weight:
  type: s4
  packing: two_per_byte_low_first
  granularity: per_channel_per_group
  group_size: 64
accumulator:
  type: i32
  overflow_policy: verifier_reject_or_split_k
requant:
  multiplier_bits: 32
  shift_max: 62
  rounding: nearest_even
  saturation: signed_i8
```

공식 참고자료

- MLIR Quant Dialect: <https://mlir.llvm.org/docs/Dialects/QuantDialect/>
- MLIR Quantization Design: <https://mlir.llvm.org/docs/Quantization/>
- MLIR TOSA Dialect: <https://mlir.llvm.org/docs/Dialects/TOSA/>
- MLIR Passes: <https://mlir.llvm.org/docs/Passes/>